



Download HTML



Download PDF

Date: Oct 2, 2025

FILTER ELEMENTS



(Narrow Down Your Locators!)

Sometimes we have **multiple elements** matching our locator, but we only want to interact with **ONE** specific element. That's where **filter()** comes to the rescue! 🎯

Filter helps us narrow down a list of elements based on specific criteria like text content or visibility!

! IMPORTANT!

`filter()` is a Locator method that returns a **NEW** filtered Locator. It's super useful when dealing with lists, dropdowns, search suggestions, and footer links!

The Problem 🤔

Scenario: You have 16 footer links, but want to click **ONLY** "Privacy Policy"

Footer Links:

[About Us](#)[Contact](#)[Privacy Policy](#)[Terms](#)[FAQ](#)[Careers](#)[Support](#)[Blog](#)[Returns](#)[Shipping](#)[Sitemap](#)[Newsletter](#)

Gift Cards

Brands

Specials

Affiliates

✗ Problem: How do we click *ONLY* the green one (Privacy Policy)?

The Solution: filter() Method! ✓

The **filter()** method narrows down elements based on conditions!



Syntax

```
locator.filter({  
  hasText: 'text to match',  
  has: anotherLocator,  
  hasNot: anotherLocator  
})
```



Key Filter Options:

hasText: Filter by partial text match

has: Filter by child element presence

hasNot: Filter by absence of child element

Use Case 1: Footer Links 🔗

Click on "Privacy Policy" from 16 footer links



Footer Links Example

```
await page.goto('https://naveenautomationlabs.com/opencart/')  
  
// Step 1: Get ALL footer links (16 elements)  
let footersEle: Locator = page.locator('footer a');
```

```
// Step 2: Filter to get ONLY "Privacy Policy" link
footersEle.filter({
  hasText: 'Privacy Policy',
  visible: true
}).click();
```

How it works:

1. Find all footer links (16 total)
2. Filter for text containing "Privacy Policy"
3. Make sure it's visible
4. Click it! 🎉

Use Case 2: Google Search Suggestions

Select a specific suggestion from Google autocomplete dropdown

Google Search Example

```
await page.goto('https://www.google.com/');

// Type search query
await page.locator('[name='q']').fill('selenium testing');

// Wait for suggestions and click specific one
await page.locator('ul[role='listbox'] li div[role='option']
  .filter({ hasText: 'websites' })
  .click();
```

What happens:

Search Suggestions appear:

- selenium testing
- selenium testing tools
- selenium testing websites ← We click this!

→ *selenium testing tutorial*

The filter finds the suggestion containing "websites" and clicks it!

Use Case 3: Flipkart Product Search

Search for "macbook" and select specific suggestion "air m3"



Flipkart Example

```
await page.goto('https://www.flipkart.com/');  
  
// Search for product  
await page.getByPlaceholder('Search for Products, Brands and  
    .fill('macbook');  
  
// Select specific suggestion  
await page.locator(`a.oleBil`)  
    .filter({ hasText: 'air m3' })  
    .click();
```



Suggestions that appear:

- *macbook pro*
- ***macbook air m3*** ← *Filtered & Clicked!*
- *macbook pro m4*
- *macbook accessories*

Use Case 4: Amazon Product Search

Search and select specific product suggestion from Amazon



Amazon Example

```
await page.goto('https://www.amazon.co.in');
```

```
// Type in search box
await page.locator('#twotabsearchtextbox').fill('macbook');

// Click on specific suggestion
await page.locator(`div[role='button']`)
  .filter({ hasText: 'pro m4 max' })
  .click();
```

Dropdown Options:

- macbook air
- macbook pro m3
- **macbook pro m4 max** ← Clicked!
- macbook charger

REMEMBER!

- `filter()` returns a *NEW* Locator (you can chain it!)
- `hasText` does *PARTIAL* match (case-sensitive!)
- Perfect for search suggestions, dropdowns, and lists

Filter Options in Detail

1 `hasText` (Most Common!)

- Filters elements containing the specified text
- Case-sensitive
- Partial match (doesn't need exact text)

```
.filter({ hasText: 'Privacy' }) // Matches "Privacy Policy"
```

2 `has` (Advanced)

→ Filters elements that contain a specific child element

```
.filter({ has: page.locator('.icon') }) // Has child with
```

3 hasNot (Exclusion)

→ Filters elements that DON'T contain a specific child element

```
.filter({ hasNot: page.locator('.disabled') }) // Exclude
```

Before vs After filter() 📊

❌ Without filter() - Complex XPath!

```
// Ugly and hard to read! 😞  
await page.locator(`//footer//a[contains(text(), 'Privacy P
```

↓ Much Better! ↓

✅ With filter() - Clean & Readable!

```
// Clean and easy to understand! 😊  
let footersEle: Locator = page.locator('footer a');  
footersEle.filter({ hasText: 'Privacy Policy' }).click();
```

❌ COMMON MISTAKES

- × Forgetting hasText is case-sensitive
- × Using exact match instead of partial match
- × Not waiting for elements to appear before filtering
- × Filtering on empty locator (no elements found)

× Using `filter()` when a simple locator would work

★ PRO TIPS!

- 💡 Use `filter()` for dynamic lists (search suggestions, dropdowns)
- 💡 Chain multiple filters if needed: `.filter().filter()`
- 💡 Combine with `first()`, `last()`, `nth()` for precision
- 💡 `hasText` is your friend for text-based filtering
- 💡 Keep it simple - use `filter()` only when necessary!

Quick Reference Card

Method	Use Case	Example
hasText	Filter by text content	<code>hasText: 'Privacy'</code>
has	Filter by child element	<code>has: page.locator('.icon')</code>
hasNot	Exclude by child element	<code>hasNot: page.locator('.disabled')</code>

✨ KEY TAKEAWAYS

- `filter()` narrows down a list of elements
- `hasText` is PERFECT for search suggestions & dropdowns
- Returns a NEW Locator (can be chained!)
- Makes code cleaner and more readable

→ Essential for handling dynamic lists! 🎯

NAVEEN AUTOMATION LABS

Your Journey to Test Automation Excellence 🎯